

Make at the center of your pipelines

Tired of playing detective with your automations? Stop guessing and start knowing. 🕵️

The context

In tools like Make, when something fails, you get a red circle. You click, try to see what data passed through that node, and cross your fingers that the log hasn't expired — or isn't clear enough. That's not engineering. That's hope.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step
from wpipe import Parallel

@step(name="branch_a")
def branch_a(data): return {"a": data["x"]}

@step(name="branch_b")
def branch_b(data): return {"b": data["y"]}

pipe = Pipeline(pipeline_name="make_to_wpipe")
pipe.set_steps([Parallel(steps=[branch_a, branch_b])])
```

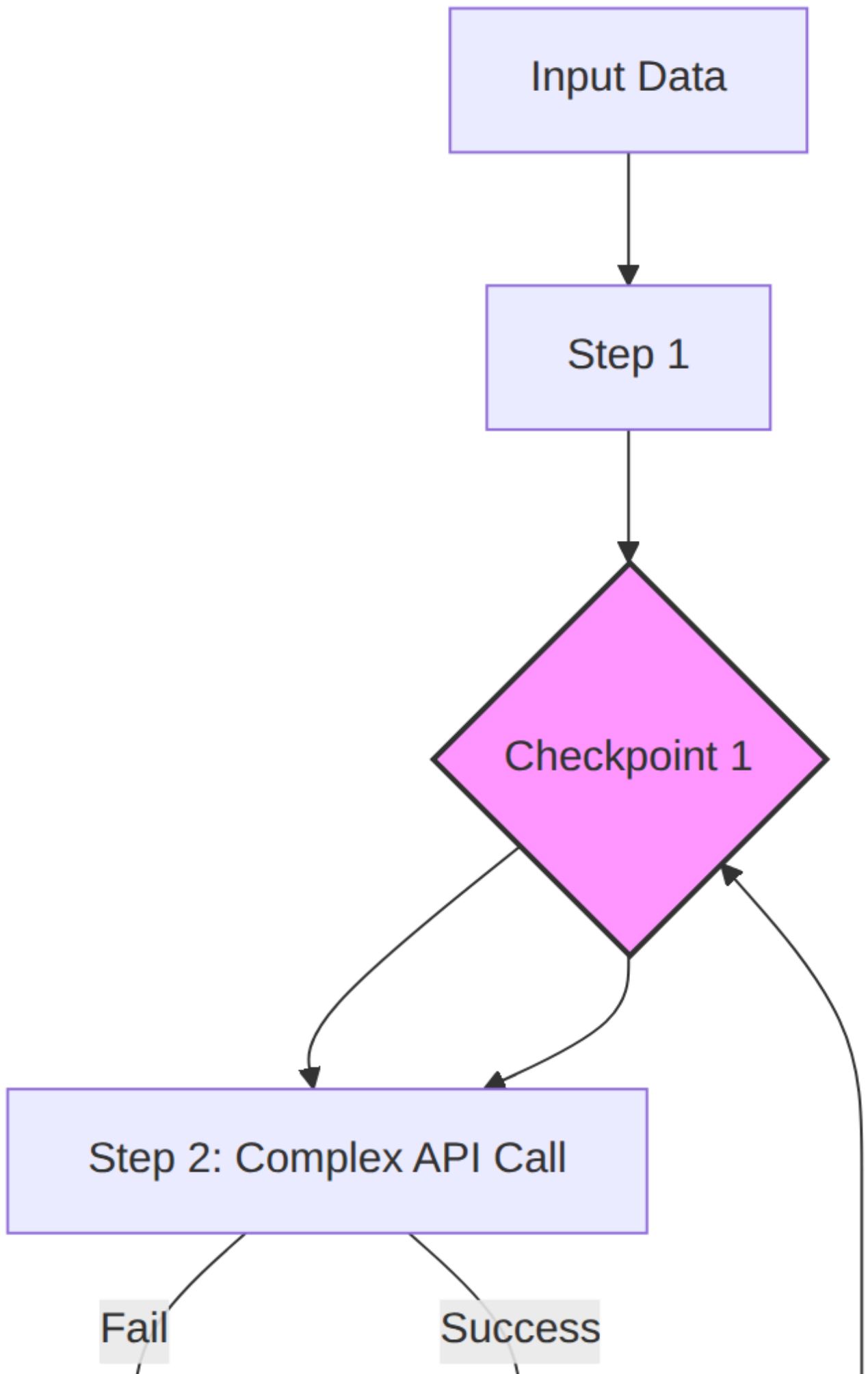
At a glance

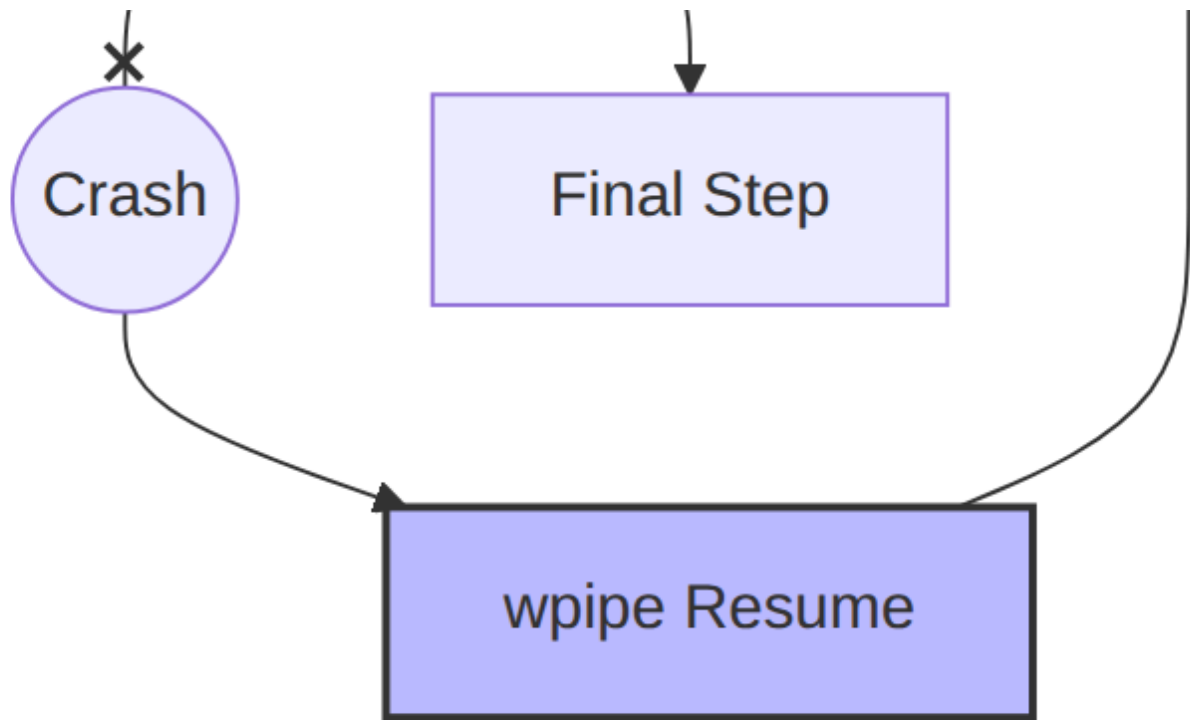
Feature	Make	wpipe
State Persistence	Ephemeral / Cloud	Persistent (Local SQLite)
Failure Recovery	Manual / Re-run	Automatic (Checkpoints)
Data Visibility	Visual inspectors	SQL Query / Web Dashboard

Feature	Make	wpipe
Retry Strategy	Basic	Advanced (Configurable per step)

Under the hood

The flow behind the post, as wpipe renders it from code.





+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Make workflows** at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.